

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed:

1. **Hash Generation using SHA-256:**
 - o Developed a Python program to compute a SHA-256 hash for any given input string using the `hashlib` library.
2. **Target Hash Generation with Nonce:**
 - o Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. **Proof-of-Work Puzzle Solving:**
 - o Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. **Merkle Tree Construction:**
 - o Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

THEORY -

1. Cryptographic Hash Functions in Blockchain

A cryptographic hash function takes input data of any size and produces a fixed-size string called a hash.

Example:

Input: "Hello"

Hash: 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969

A good cryptographic hash function has these properties:

- Deterministic — same input → same hash.
- One-way — difficult to recover the original input from the hash.
- Collision resistant — difficult to find two different inputs with the same hash.
- Avalanche effect — a tiny change in input produces a completely different hash.
- Fast to calculate — hashing should be computationally efficient.

Role in Blockchain

Hash functions are used for:

1. Block identification
2. Linking blocks together
3. Transaction integrity
4. Merkle Trees
5. Proof of Work
6. Digital signatures and addresses in some blockchain systems

For example:

Block 1

↓ hash

Block 2

↓ hash

Block 3

Each block contains information related to the previous block's hash. If someone modifies an old block, its hash changes, breaking the chain.

2. What is a Merkle Tree?

A Merkle Tree is a tree data structure used to efficiently and securely summarize a large number of transactions using hashes.

It is also called a Hash Tree.

Suppose a block contains four transactions:

Transaction A
Transaction B
Transaction C
Transaction D

First, each transaction is hashed:

$A \rightarrow \text{Hash}(A)$
 $B \rightarrow \text{Hash}(B)$
 $C \rightarrow \text{Hash}(C)$
 $D \rightarrow \text{Hash}(D)$

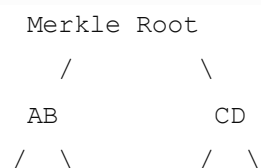
Then hashes are combined in pairs:

$\text{Hash}(\text{Hash}(A) + \text{Hash}(B)) \rightarrow AB$
 $\text{Hash}(\text{Hash}(C) + \text{Hash}(D)) \rightarrow CD$

Finally:

$\text{Hash}(AB + CD) \rightarrow \text{Merkle Root}$

The Merkle Root is stored in the block header.



A B C D

3. How Does a Merkle Tree Work?

The process is:

Step 1 — Hash transactions

$A \rightarrow H(A)$

$B \rightarrow H(B)$

$C \rightarrow H(C)$

$D \rightarrow H(D)$

Step 2 — Combine adjacent hashes

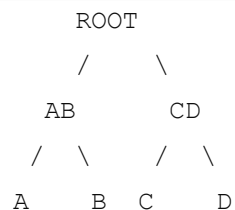
$H(A) + H(B) \rightarrow H(AB)$

$H(C) + H(D) \rightarrow H(CD)$

Step 3 — Continue upward

$H(AB) + H(CD) \rightarrow \text{Merkle Root}$

So the structure becomes:



If Transaction B changes, its hash changes. That causes `AB` to change, which causes the Merkle Root to change.

Therefore, the Merkle Root acts as a compact fingerprint of all transactions in the block.

4. Benefits of a Merkle Tree

1. Efficient verification

You don't need every transaction to verify that a particular transaction belongs to a block.

A Merkle Proof can prove membership using only a small number of hashes.

For example, to prove that B exists:

```
B
+
A
→ AB

AB
+
CD
→ ROOT
```

You only need the relevant hashes rather than the entire transaction list.

2. Detects tampering

Changing even one transaction changes its hash and ultimately changes the Merkle Root.

3. Saves storage and bandwidth

Nodes can use compact proofs rather than transferring the entire set of transactions.

4. Fast verification

Merkle proofs can be verified in approximately $O(\log n)$ hash operations for n transactions.

5. Scalability

Merkle Trees allow blockchain systems to handle and verify large numbers of transactions efficiently.

5. Use Cases of Merkle Trees

1. Bitcoin

Bitcoin uses Merkle Trees to summarize the transactions in each block.

The Merkle Root is included in the block header.

2. Ethereum

Ethereum uses Merkle/Patricia-tree-based structures to efficiently represent and verify blockchain state and transaction-related data.

3. Lightweight / SPV Clients

Simplified Payment Verification (SPV) clients can verify that a transaction is included in a block without downloading the entire blockchain.

4. Data Integrity

Merkle Trees can prove that data has not been modified.

5. Distributed Systems

They can efficiently compare large datasets between different systems.

6. File and Version Verification

Merkle-tree structures can be used to verify that particular pieces of data belong to a larger dataset.

6. Cryptographic Puzzle & Golden Nonce

A cryptographic puzzle is a computational problem designed to be difficult to solve but easy for others to verify.

In Proof of Work, miners repeatedly change a value called a nonce and calculate the block hash.

For example:

Block Data + Nonce

↓

SHA-256

↓

Hash

The miner searches for a nonce such that the resulting hash satisfies a particular condition, such as being below a target.

Example conceptually:

```
Nonce = 1 → invalid
Nonce = 2 → invalid
Nonce = 3 → invalid
...
Nonce = 728391 → VALID
```

Golden Nonce

The golden nonce generally refers to the nonce value that produces a hash satisfying the required Proof-of-Work condition.

For example:

```
Block data + Golden Nonce
      ↓
    SHA-256
      ↓
000000000000000000000000abc123...
      ↑
    satisfies target
```

The important point is that the miner normally cannot calculate the nonce directly. It must search through possible nonce values, making Proof of Work computationally expensive.

Program and Output

Program #1: Python program that uses the hashlib library to create the hash of a given string:

```
import hashlib                                     #error
statement

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()          #error
statement
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)                       #error
statement
```

Output :

Enter a string: Siddharth

Hash: 612f14433c2d02e3d4360665272589a6ca9c593b7527ec2d9543376d812f2008

Program #2: Program to generate required target hash with input string and nonce

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")
```

```

# Concatenate the string and nonce
hash_string = input_string + nonce
Error

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)
Error

Enter a string: Siddharth
Enter the nonce: 1195820
Hash Code:
00000972f021bc657d44277d090bdcfa8445b369f9a719306ffef70db2fdb3ff

```

Program #3: Python code for Solving Puzzle for leading zeros with expected nonce and given string

```

import hashlib
def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros
    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
    #Error
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
    #Error
            print("Hash:", hash_code)
            return nonce

```

```

        nonce += 1
# Get user input
input_string = "Siddharth"
num_zeros = 5

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
#Error

Hash: 00000972f021bc657d44277d090bdcfa8445b369f9a719306ffef70db2fdb3ff
Input String: Siddharth
Leading Zeros: 5
Expected Nonce: 1195820

```

Program 4: Generating Merkle Tree for given set of Transactions

```

import hashlib
def hash_transaction(transaction):
    return hashlib.sha256(transaction.encode('utf-8')).hexdigest()
def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None
    # Step 1: Hash each transaction individually
    transactions = [hash_transaction(tx) for tx in transactions]

    # Print individual transaction hashes
    print("\nIndividual Transaction Hashes:")
    for i, tx_hash in enumerate(transactions):
        print(f"Transaction {i + 1}: {tx_hash}")

    # Step 2: Keep combining hashes until only one remains

```

```

while len(transactions) > 1:

    # If odd number of hashes, duplicate the last one
    if len(transactions) % 2 != 0:
        transactions.append(transactions[-1])

    new_transactions = []

    # Combine pairs and hash them
    for i in range(0, len(transactions), 2):
        combined = transactions[i] + transactions[i + 1]
        hash_combined = hashlib.sha256(
            combined.encode('utf-8')
        ).hexdigest()
        new_transactions.append(hash_combined)
    transactions = new_transactions
    return transactions[0]

# Transactions
transactions = [
    "Siddharth -> Gore: $300",
    "Rutuja -> Rutuja: $200",
    "Afnan -> Ved: $300",
    "Ved -> Bob: $30000"
]

# Build Merkle Tree
merkle_root = build_merkle_tree(transactions)

print("\nMerkle Root:", merkle_root)

```

\

Output :

Transaction Hashes:

Transaction 1:

f16d247a78d76de30d247f5ec95f36cf31301eb54242a476a9a617f9fd6f5d65

Transaction 2:

fc8102cd3cedc9fdae34f71ad239a15fc6c641dce92d314aa6327ba0e00fbc29

Transaction 3:

f624e157490da71806d3e9d0911c3f5fa53f69edf1b53566fea0ef82a2a6ce3c

Transaction 4:

6f881bd56fd9d5cb5ec7ce2e4fb5bd5247d387942280990ee1d260cee02bc1f7

Transaction 5:

3cb00b6c868029bce21986e2b845a2d47788fbb029bd12b7f6ea9e253c69e6f2

Next Level:

98f7de93f1aab9961a8deb75a293c8d6245af6591b4833a105f62c4d32af5e8c
c5b0544ec31f6faf0e5f5f658287245a6a6231ea359cbd4ddbde9b8c6fab072e
9295aaa068e5269a09c74898234d0a2f28460d47b1fa5df08e439de44222651e

Next Level:

cf66ecbdfdd43e9cfc8c8d8b4c7417403e66cb70c4fa7e6ed9cab0d39e5c356a
192f5368654b32c94fedaa68e3f106c425cf3ab1ec5b8465fcc33acd30140f65

Next Level:

76f05a112ac75e1ee1b0c2d1e91a17b11ea8ccccdd48cf06e2d87c85e747fd93

=====

Merkle Root:

76f05a112ac75e1ee1b0c2d1e91a17b11ea8ccccdd48cf06e2d87c85e747fd93

=====

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.